

Homework 3 Report: Java Shared Memory Performance Races

Computer Science 131
University of California Los Angeles

Abstract

The "synchronized" keyword is frequently used in multi-threaded Java applications to prevent data races and thus produce intended results. However, it works by implementing a lock on a resource, and when a thread uses that resource, it becomes locked to other threads. Not only does implementing this lock have significant overhead, it also causes threads to wait for long periods of time and is thus a major efficiency bottleneck. In the context of applications where efficiency is far more important than accuracy, such as the context of GDI in this assignment, unsafe alternatives are worth exploring. Here, I explore two alternatives to synchronized: one that uses the volatile keyword to ensure a visibility guarantee (AcmeSafeState) and another that puts no restrictions on threads/memory accesses at all (Unsynchronized state). Results reveal that when using few threads, it is better to use unsynchronized or other unsafe methods. However, when using many threads, the efficiency difference between synchronized and unsynchronized is very little, and it would be better to use the synchronized method so as to guarantee perfect results.

1 Introduction

I used a simulation program on an array of longs to test the relative speeds of different multithreading approaches in Java. Three approaches were compared: using the synchronized keyword to make swaps safer, using the volatile keyword to make the array's memory visible to all threads, and unrestricted access. I tested the simulations on arrays of sizes 5 entries and 100 entries, with thread counts of 1, 8 and 40, and using both platform and virtual threads. Virtual threads are threads that are managed by the Java Virtual Machine (JVM) instead of the operating system. My testing platform has the following statistics: openjdk version "23.0.2" CPU: Intel Xeon Silver 4116 @ 2.10 GHz Cores visible to OS: 4 Cache: 16,896 KB (16.5 MB) L3 cache, 64-byte cache lines Architecture: x86-64 with AVX-512 support Total RAM: 32.5 GB

2 Problems

Some problems that could interfere with the accuracy of these measurements include SEASnet being shared machines. This can cause timings to vary and results to not be perfectly reproducible because results can depend on how much load the servers are currently under and how many other users are concurrently also using the CPU. I mitigated this problem by running all of my tests within roughly the same 20-minute timeframe, so as to minimize server-based variance. Another problem is that Java has significant startup overhead for setting up threads and running code. I mitigated this problem by running the NullState tests, which functionally does nothing but can be used to provide a helpful baseline to compare the other tests to. It essentially times how long it takes to do the setup and thus finds the "fastest" possible result, which is a good metric for how much startup overhead is in each test. One more problem is that the accuracy metric is not perfect. An output sum of 0 does not necessarily mean everything executed without error. The errors could have cancelled out so this is not a great metric for comparing accuracy. This problem can be mitigated by running a high magnitude of operations, such as 100 million. It's statistically impossible for 100 million unsafe operations to all cancel each other out. However, this does not improve the quality of the metric - if two error measurements are within the same order of magnitude, comparing them is flawed because of statistical anomalies. We can only say one approach is definitely safer than another if the difference between their error measurements is greater than an order of magnitude.

3 Measurements

Table 1 shows average swap time (ns) across all configurations. Table 2 shows the magnitude of reliability failures (output sum $\neq$ 0) where they occur.

Table 1: Average swap time (ns) per configuration.

Threads	Size	Null	Sync	Acme	Unsync
<i>Platform threads</i>					
1	5	12.7	35.6	14.7	13.6
1	100	14.0	34.0	15.6	15.0
8	5	35.9	341	257	161
8	100	35.5	301	299	269
40	5	159	1746	1248	1057
40	100	150	1468	1446	1404
<i>Virtual threads</i>					
1	5	13.0	35.4	14.5	13.9
1	100	14.6	35.0	15.9	15.1
8	5	32.5	343	263	169
8	100	36.6	328	335	303
40	5	175	1629	1435	882
40	100	171	1659	1570	1497

Table 2: Output sum mismatch magnitude where reliability failures occur. Null and Synchronized never fail; AcmeSafe and Unsynchronized always fail with ≥ 2 threads.

Threads	Size	Acme	Unsync
<i>Platform threads</i>			
8	5	443114	23696
8	100	21866	11045
40	5	399884	7894
40	100	13494	4211
<i>Virtual threads</i>			
8	5	224129	64931
8	100	19559	14456
40	5	244724	53022
40	100	21394	12941

4 Analysis

The results that I observed that were not surprising is that in almost every case, SynchronizedState is the slowest approach and UnsynchronizedState is the fastest, with AcmeSafeState (which I implemented as the same as UnsynchronizedState, but the array of longs is volatile) being somewhere in between. The reason for this must be because UnsynchronizedState and AcmeSafeState break sequential consistency, while SynchronizedState doesn't. This means threads in SynchronizedState have to wait a lot at times to update the array, in AcmeSafeState they have to wait a little, and in UnsynchronizedState they don't have to wait at all. The time spent waiting because a thread can't update the array safely is time that is wasted. This does mean that UnsynchronizedState and AcmeSafeState do not necessarily produce the correct result, but the magnitude of the error is in the ten thousands per 100 million calculations, which is an approximately 0.01% margin of error. For an application like this where a little breakage is tolerable

as long as the program runs faster, this is an acceptable margin of error. Another interesting result that I observed is that when using only a single thread, the Synchronized approach is still considerably slower than the other two approaches, who are very close to the baseline of null. Synchronized is nearly 2.5x slower in these instances than either Unsynchronized or Acme. Therefore, the synchronize keyword must have a fixed overhead cost that slows down the program. Another interesting result is that, in the particular case of using a 100-entry array with 40 threads, the gap between the classes is very small: only 22 more nanoseconds than Acme and only 64 nanoseconds more than Unsynchronized. This is also true for the 100-entry array with 8 threads case - the difference between the approaches is very small. The pattern seems to be that the gap gets smaller as threads get added. This is likely because when we introduce race conditions, conflict between threads can also slow the program down, and more threads means more conflicts. I also found that the volatile approach is actually less safe than the unsynchronized approach, usually having a higher error rate for every case, sometimes by an order of magnitude. I also noticed that the difference between the performance of platform threads and virtual threads is really insignificant.

5 Conclusion

The data and results lead me to conclude that using the unsynchronized approach is best when using small numbers of threads because synchronized is much slower than unsynchronized in this context. When using a large number of threads, however, the difference in performance between is so little it's insignificant so it might be worth to use synchronize to guarantee perfect results. I also found that using virtual threads as opposed to platform threads has almost no impact on performance, though I admit this could be impacted by differences in machine specs and operating system.

Appendix

A Raw Output

5 entries, 1 platform thread:

Null:

sh-5.1\$ time timeout 3600 java UnsafeMemory Platform 5 Null 1 100000000

Total real time 1.26782 s

Average real swap time 12.6782 ns

real 0m1.372s

user 0m1.309s

sys 0m0.046s

Synchronized:

sh-5.1\$ time timeout 3600 java UnsafeMemory Platform 5 Synchronized 1 100000000

Total real time 3.56436 s

Average real swap time 35.6436 ns

real 0m3.657s

user 0m3.616s

sys 0m0.036s

AcmeSafe:

sh-5.1\$ time timeout 3600 java UnsafeMemory Platform 5 AcmeSafe 1 100000000

Total real time 1.47365 s

Average real swap time 14.7365 ns

real 0m1.570s

user 0m1.550s

sys 0m0.033s

Unsynchronized:

sh-5.1\$ time timeout 3600 java UnsafeMemory Platform 5 Unsynchronized 1 100000000

Total real time 1.35586 s

Average real swap time 13.5586 ns

real 0m1.454s

user 0m1.440s

sys 0m0.030s

100 entries, 1 platform thread:

Null:

sh-5.1\$ time timeout 3600 java UnsafeMemory Platform 100 Null 1 100000000

Total real time 1.39522 s

Average real swap time 13.9522 ns

real 0m1.492s

user 0m1.451s

sys 0m0.039s

Synchronized:

sh-5.1\$ time timeout 3600 java UnsafeMemory Platform 100 S

Total real time 3.40440 s

Average real swap time 34.0440 ns

real 0m3.511s

user 0m3.493s

sys 0m0.033s

AcmeSafe:

sh-5.1\$ time timeout 3600 java UnsafeMemory Platform 100 A

Total real time 1.55872 s

Average real swap time 15.5872 ns

real 0m1.658s

user 0m1.646s

sys 0m0.037s

Unsynchronized:

sh-5.1\$ time timeout 3600 java UnsafeMemory Platform 100 U

Total real time 1.49860 s

Average real swap time 14.9860 ns

real 0m1.598s

user 0m1.579s

sys 0m0.044s

5 entries, 8 platform threads:

Null:

sh-5.1\$ time timeout 3600 java UnsafeMemory Platform 5 Nul

Total real time 0.449127 s

Average real swap time 35.9302 ns

real 0m0.551s

user 0m1.535s

sys 0m0.044s

Synchronized:

sh-5.1\$ time timeout 3600 java UnsafeMemory Platform 5 Syn

Total real time 4.26413 s

Average real swap time 341.131 ns

real 0m4.360s

user 0m6.493s

sys 0m0.200s

AcmeSafe:

sh-5.1\$ time timeout 3600 java UnsafeMemory Platform 5 Acme

Total real time 3.21200 s

Average real swap time 256.960 ns

output sum mismatch (-443114 != 0)

real 0m3.302s

user 0m12.030s

```

sys      0m0.044s
Null:
sh-5.1$ time timeout 3600 java UnsafeMemory Platform 5 Null
Total real time 0.397912 s
Average real swap time 161.190 ns
output sum mismatch (23696 != 0)
real      0m2.181s
user      0m7.417s
sys      0m0.058s
100 entries, 8 platform threads:
Null:
sh-5.1$ time timeout 3600 java UnsafeMemory Platform 5 Null
Total real time 0.443528 s
Average real swap time 35.4823 ns
real      0m0.536s
user      0m1.723s
sys      0m0.039s
Synchronized:
sh-5.1$ time timeout 3600 java UnsafeMemory Platform 5 Synchronized 8 100000000
Total real time 3.76718 s
Average real swap time 301.374 ns
real      0m3.879s
user      0m5.165s
sys      0m0.137s
AcmeSafe:
sh-5.1$ time timeout 3600 java UnsafeMemory Platform 5 AcmeSafe 8 100000000
Total real time 3.74018 s
Average real swap time 299.215 ns
output sum mismatch (-21866 != 0)
real      0m3.840s
user      0m11.775s
sys      0m0.098s
Unsynchronized:
sh-5.1$ time timeout 3600 java UnsafeMemory Platform 5 Unsynchronized 8 100000000
Total real time 3.36294 s
Average real swap time 269.035 ns
output sum mismatch (-11045 != 0)
real      0m3.543s
user      0m9.526s
sys      0m0.124s
5 entries, 40 platform threads:
Null:
sh-5.1$ time timeout 3600 java UnsafeMemory Platform 5 Null
Total real time 0.397912 s
Average real swap time 161.190 ns
output sum mismatch (23696 != 0)
real      0m0.489s
user      0m1.558s
sys      0m0.055s
Synchronized:
Total real time 4.36418 s
Average real swap time 1745.67 ns
real      0m4.480s
user      0m6.507s
AcmeSafe:
Total real time 3.11926 s
Average real swap time 1247.70 ns
output sum mismatch (-399884 != 0)
real      0m3.213s
user      0m9.519s
Unsynchronized:
sh-5.1$ time timeout 3600 java UnsafeMemory Platform 5 Unsynchronized 8 100000000
Total real time 2.64198 s
Average real swap time 1056.79 ns
output sum mismatch (-7894 != 0)
real      0m2.738s
user      0m0.081s
sys      0m0.093s
100 entries, 40 platform threads:
Null:
sh-5.1$ time timeout 3600 java UnsafeMemory Platform 100 Null
Total real time 0.375006 s
Average real swap time 150.003 ns
Synchronized:
sh-5.1$ time timeout 3600 java UnsafeMemory Platform 100 Synchronized 8 100000000
Total real time 3.67077 s
Average real swap time 1468.31 ns
real      0m3.776s
user      0m4.804s

```

```

sys      0m0.191s
AcmeSafe:
sh-5.1$ time timeout 3600 java UnsafeMemory Platform 100 AcmeSafe 40 100000000
Total real time 3.61380 s
Average real swap time 1445.52 ns
output sum mismatch (-13494 != 0)

real      0m3.712s
user      0m11.507s
sys       0m0.045s

Unsyncronized:
sh-5.1$ time timeout 3600 java UnsafeMemory Platform 100 AcmeSafe 40 100000000
Total real time 3.50962 s
Average real swap time 1403.85 ns
output sum mismatch (-4211 != 0)

real      0m3.607s
user      0m12.452s
sys       0m0.087s

Virtual Threads:

5 entries, 1 virtual thread:

Null:
sh-5.1$ time timeout 3600 java UnsafeMemory Virtual 5 Null 1 100000000
Total real time 1.29585 s
Average real swap time 12.9585 ns

real      0m1.406s
user      0m1.377s
sys       0m0.040s

Synchronized:
sh-5.1$ time timeout 3600 java UnsafeMemory Virtual 5 Synchronized 1 100000000
Total real time 3.53978 s
Average real swap time 35.3978 ns

real      0m3.654s
user      0m3.578s
sys       0m0.040s

AcmeSafe:
sh-5.1$ time timeout 3600 java UnsafeMemory Virtual 5 AcmeSafe 1 100000000
Total real time 1.44855 s
Average real swap time 14.4855 ns

real      0m1.559s
user      0m1.540s
sys       0m0.038s

Unsyncronized:

sh-5.1$ time timeout 3600 java UnsafeMemory Virtual 5 UnsafeMemory Virtual 5 Unsy
Total real time 1.39232 s
Average real swap time 13.9232 ns

real      0m1.492s
user      0m1.476s
sys       0m0.043s

100 entries, 1 virtual thread:

Null:
sh-5.1$ time timeout 3600 java UnsafeMemory Virtual 100 Null 1 100000000
Total real time 1.46330 s
Average real swap time 14.6330 ns

real      0m1.573s
user      0m1.559s
sys       0m0.040s

Synchronized:
sh-5.1$ time timeout 3600 java UnsafeMemory Virtual 100 Synchronized 1 100000000
Total real time 3.50344 s
Average real swap time 35.0344 ns

real      0m3.606s
user      0m3.529s
sys       0m0.063s

AcmeSafe:
sh-5.1$ time timeout 3600 java UnsafeMemory Virtual 100 AcmeSafe 1 100000000
Total real time 1.58853 s
Average real swap time 15.8853 ns

real      0m1.708s
user      0m1.684s
sys       0m0.044s

Unsyncronized:
sh-5.1$ time timeout 3600 java UnsafeMemory Virtual 100 UnsafeMemory Virtual 100 Un
Total real time 1.50752 s
Average real swap time 15.0752 ns

real      0m1.616s
user      0m1.581s
sys       0m0.047s

5 entries, 8 virtual threads:

Null:
sh-5.1$ time timeout 3600 java UnsafeMemory Virtual 5 Null 1 100000000
Total real time 0.406644 s
Average real swap time 32.5315 ns

real      0m0.515s

```

```

user      0m1.567s
sys       0m0.043s

Average real swap time 334.904 ns
output sum mismatch (-19559 != 0)

Synchronized:
Total real time 4.29086 s
Average real swap time 343.269 ns

real      0m4.396s
user      0m7.387s
sys       0m1.614s

AcmeSafe:
sh-5.1$ time timeout 3600 java UnsafeMemory Virtual 5 AcmeSafe 8 100000000
Total real time 3.28473 s
Average real swap time 262.779 ns
output sum mismatch (-224129 != 0)

real      0m3.425s
user      0m11.521s
sys       0m0.067s

Unsynchronized:
sh-5.1$ time timeout 3600 java UnsafeMemory Virtual 5 Unsynchronized 8 100000000
Total real time 2.10973 s
Average real swap time 168.779 ns
output sum mismatch (64931 != 0)

real      0m2.214s
user      0m8.162s
sys       0m0.048s

100 entries, 8 virtual threads:

Null:
sh-5.1$ time timeout 3600 java UnsafeMemory Virtual 5 Null 8 100000000
Total real time 0.456870 s
Average real swap time 36.5496 ns

real      0m0.561s
user      0m1.798s
sys       0m0.046s

Synchronized:
sh-5.1$ time timeout 3600 java UnsafeMemory Virtual 5 Synchronized 8 100000000
Total real time 4.10269 s
Average real swap time 328.215 ns

real      0m4.208s
user      0m6.780s
sys       0m1.291s

AcmeSafe:
sh-5.1$ time timeout 3600 java UnsafeMemory Virtual 100 AcmeSafe 8 100000000
Total real time 4.18630 s

Average real swap time 334.904 ns
output sum mismatch (-19559 != 0)

real      0m4.292s
user      0m16.153s
sys       0m0.041s

Unsynchronized:
sh-5.1$ time timeout 3600 java UnsafeMemory Virtual 100 Unsynchronized 8 100000000
Total real time 3.78982 s
Average real swap time 303.185 ns
output sum mismatch (-14456 != 0)

real      0m3.890s
user      0m14.618s
sys       0m0.087s

5 entries, 40 virtual threads:

Null:
sh-5.1$ time timeout 3600 java UnsafeMemory Virtual 5 Null 40 100000000
Total real time 0.437186 s
Average real swap time 100.0000075 ns

real      0m0.540s
user      0m1.611s
sys       0m0.040s

Synchronized:
sh-5.1$ time timeout 3600 java UnsafeMemory Virtual 5 Synchronized 40 100000000
Total real time 4.07234 s
Average real swap time 1628.94 ns

real      0m4.189s
user      0m11.400s
sys       0m1.361s

AcmeSafe:
sh-5.1$ time timeout 3600 java UnsafeMemory Virtual 5 AcmeSafe 40 100000000
Total real time 3.58689 s
Average real swap time 1434.76 ns
output sum mismatch (-244724 != 0)

Synchronized:
sh-5.1$ time timeout 3600 java UnsafeMemory Virtual 100 Synchronized 40 100000000
Total real time 4.10269 s
Average real swap time 328.215 ns

real      0m4.208s
user      0m6.780s
sys       0m1.291s

AcmeSafe:
sh-5.1$ time timeout 3600 java UnsafeMemory Virtual 100 AcmeSafe 8 100000000
Total real time 4.18630 s

```

```

user      0m8.348s
sys       0m0.059s

100 entries, 40 virtual threads:

Null:
sh-5.1$ time timeout 3600 java UnsafeMemory Virtual 1000000000
Total real time 0.426752 s
Average real swap time 170.701 ns

real      0m0.543s
user      0m1.530s
sys       0m0.046s

Synchronized:
sh-5.1$ time timeout 3600 java UnsafeMemory Virtual 1000000000
Total real time 4.14805 s
Average real swap time 1659.22 ns

real      0m4.273s
user      0m6.719s
sys       0m1.467s

AcmeSafe:
sh-5.1$ time timeout 3600 java UnsafeMemory Virtual 1000000000
Total real time 3.92597 s
Average real swap time 1570.39 ns
output sum mismatch (-21394 != 0)

real      0m4.041s
user      0m13.022s
sys       0m0.107s

Unsynchronized:
sh-5.1$ time timeout 3600 java UnsafeMemory Virtual 1000000000
Total real time 3.74160 s
Average real swap time 1496.64 ns
output sum mismatch (-12941 != 0)

real      0m3.872s
user      0m11.211s
sys       0m0.096s

CPU and Memory Info:
sh-5.1$ /proc/cpuinfo
sh: /proc/cpuinfo: Permission denied
sh-5.1$ cat /proc/cpuinfo
processor      : 0
vendor_id     : GenuineIntel
cpu family    : 6
model         : 85
model name    : Intel(R) Xeon(R) Silver 4116 CPU @ 2.10GHz
stepping      : 4
microcode     : 0xffffffff

processor      : 2
vendor_id     : GenuineIntel
cpu family    : 6
model         : 85
model name    : Intel(R) Xeon(R) Silver 4116 CPU @ 2.10GHz
stepping      : 4
microcode     : 0xffffffff

cpu MHz       : 2095.075
cache size    : 16896 KB
physical id    : 0
siblings      : 4
core id       : 0
cpu cores     : 4
initial apicid : 0
fpu           : yes
fpu_exception : yes
cpuid level   : 21
wp            : yes
flags         : fpu vme de pse tsc msr pae mce cx8 apic
bugs          : cpu_meltdown spectre_v1 spectre_v2 spec_
bogomips      : 4190.15
bogoimips     : 4190.15
cache_alignm   : 64
address sizes  : 44 bits physical, 48 bits virtual
power managem   :

processor      : 1
vendor_id     : GenuineIntel
cpu family    : 6
model         : 85
model name    : Intel(R) Xeon(R) Silver 4116 CPU @ 2.10GHz
stepping      : 4
microcode     : 0xffffffff
cpu MHz       : 2095.075
cache size    : 16896 KB
physical id    : 0
siblings      : 4
core id       : 1
cpu cores     : 4
apicid        : 1
initial apicid : 140
fpu           : yes
fpu_exception : yes
cpuid level   : 21
wp            : yes
flags         : fpu vme de pse tsc msr pae mce cx8 apic
bugs          : cpu_meltdown spectre_v1 spectre_v2 spec_
bogomips      : 4190.15
bogoimips     : 4190.15
cache_alignm   : 64
address sizes  : 44 bits physical, 48 bits virtual
power managem   :

```

```

cpu MHz      : 2095.075      Active:      9747004 kB
cache size   : 16896 KB      Inactive:    18500264 kB
physical id   : 0            Active(anon): 2716384 kB
siblings     : 4            Inactive(anon): 94772 kB
core id      : 2            Active(file): 7030620 kB
cpu cores    : 4            Inactive(file): 18405492 kB
apicid       : 2            Unevictable: 3136 kB
initial apicid : 2          Mlocked:     64 kB
fpu          : yes          SwapTotal:   8388604 kB
fpu_exception : yes        SwapFree:    8388336 kB
cpuid level  : 21          Zswap:      0 kB
wp           : yes         Zswapped:    0 kB
flags        : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca 300mb pat pse36 clflush mmx fxsr sse
bugs         : cpu_meltdown spectre_v1 spectre_v2 SpectreBypass 11tfo0mb swapgs taa itlb_multihit mmio_
bogomips     : 4190.15      AnonPages:   2756736 kB
clflush size : 64           Mapped:      621656 kB
cache_alignment : 64        Shmem:       26392 kB
address sizes : 44 bits physical, 48 bits virtual KReclaimable: 1866188 kB
power management:          Slab:            2213184 kB
                               SReclaimable: 1866188 kB
                               SUnreclaim: 346996 kB
processor     : 3            KernelStack: 13648 kB
vendor_id    : GenuineIntel PageTables:   52560 kB
cpu family   : 6            SecPageTables: 0 kB
model        : 85           2SL0G8ztable: 0 kB
model name   : Intel(R) Xeon(R) Silver 4116 CPU @2.10GHz
stepping     : 4            Bounce:      0 kB
microcode    : 0xffffffff   WritebackTmp: 0 kB
cpu MHz      : 2095.075     CommitLimit: 24653256 kB
cache size   : 16896 KB     Committed_AS: 4681580 kB
physical id   : 0           VmallocTotal: 34359738367 kB
siblings     : 4           VmallocUsed: 48912 kB
core id      : 3           VmallocChunk: 0 kB
cpu cores    : 4           Percpu:      245760 kB
apicid       : 3           HardwareCorrupted: 0 kB
initial apicid : 3         AnonHugePages: 958464 kB
fpu          : yes         ShmemHugePages: 0 kB
fpu_exception : yes        ShmemPmdMapped: 0 kB
cpuid level  : 21          FileHugePages: 0 kB
wp           : yes         FilePmdMapped: 0 kB
flags        : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca 300mb pat pse36 clflush mmx fxsr sse
bugs         : cpu_meltdown spectre_v1 spectre_v2 SpectreBypass 11tfo0mb swapgs taa itlb_multihit mmio_
bogomips     : 4190.15      Unaccepted: 0 kB
clflush size : 64           HugePages_Total: 0
cache_alignment : 64        HugePages_Free: 0
address sizes : 44 bits physical, 48 bits virtual HugePages_Rsvd: 0
power management:          HugePages_Surp: 0
                               Hugepagesize: 2048 kB
                               Hugetlb:      0 kB
sh-5.1$ cat /proc/meminfo   DirectMap4k: 392596 kB
MemTotal:      32529308 kB   DirectMap2M: 20578304 kB
MemFree:       1642164 kB    DirectMap1G: 13631488 kB
MemAvailable:  28483836 kB
Buffers:       358168 kB
Cached:        25104200 kB
SwapCached:    4 kB

```